# UNI-APP 原生插件 SDK 接口文档 V2.0.2

## 文档修改记录

| 序号 | 版本号 | 修改内容 | 文档修改者 | 修改日期 |
| --- | --- | --- | --- | --- |
| 1 | v2.0.0 | 文档建立 | 张彬 | 2025/09/18 |
| 2 | v2.0.1 | 功能优化<br>1. 文本绘制恢复粗体、斜体、下划线支持<br>2. 恢复蓝牙搜索时长5s<br>3. 恢复连接接口到指定线程执行，避免上层调用连接时页面卡死<br>4. 增加蓝牙搜索结果的deviceType字段<br>5. DEMO化拆分Index.vue为多个职责单一的组件，便于维护+复用<br>6. DEMO支持模板生成工具生成的JSON打印<br>7. DEMO封装使用蓝牙搜索结果的deviceType字段，不再通过搜索方式判断 | 张彬 | 2025/12/25 |
| 3 | v2.0.1 | 功能优化 | 张彬 | 2026/03/02 |

| | | | | |
|---|---|---|---|---|
| | | 1. Demo中"printDensity"全部修改为"density",避免造成startJob传参误导，包含变量定义、函数参数传递、组件参数传递，quickStart示例<br>2. 文档绘制参数中部分旋转参数单词拼写错误"roato"修正为"rotato"<br>3. 从错误日志中移除冗余的error对象 | | |
| 4 | v2.0.2 | BUG修复<br>1. 修正DEMO中API文件drawLabelImage函数的注释标注错误：当前参数图像数据对应的注释标记为params.value，统一将注释说明及关联文档中所有引用该参数的表述更正为params.imageData，确保接口文档与代码参数命名规范保持一致性。<br>功能优化<br>1. 更新Android侧依赖的原生SDK以在64 位设备上支持 16 KB 的页面大小。 | 张彬 | 2026/05/15 |

## DEMO 目录结构

代码块

```
UTSSample/
├── App.vue                          # 应用入口文件
├── main.js                          # Vue 应用实例创建
├── manifest.json                    # UniApp 应用配置
├── pages.json                       # 页面路由配置
├── index.html                       # H5 入口文件
├── uni.scss                         # 全局样式变量
├── uni.promisify.adaptor.js        # Promise 适配器
├── pages/
│   └── index/
│       ├── index.vue               # 主页面
│       └── index.css               # 主页面样式
├── components/                     # 业务组件
│   ├── device/                     # 设备管理组件
│   │   ├── DeviceConnection.vue    # 设备连接组件
│   │   ├── DeviceList.vue          # 设备列表组件
│   │   ├── DeviceStatus.vue        # 设备状态显示组件
│   │   ├── WifiConfigDialog.vue    # WiFi 配置对话框
│   │   ├── composables/            # 设备相关组合式函数
│   │   │   └── useDeviceSearch.js  # 设备搜索逻辑
│   │   └── shared/                 # 设备共享资源
│   │       └── device-common.css   # 设备组件通用样式
│   ├── print/                      # 打印功能组件
│   │   ├── PrintFunctions.vue      # 打印功能组件
│   │   ├── PrintActionButtons.vue  # 打印操作按钮
│   │   ├── PrintTypeSelector.vue   # 打印类型选择器
│   │   ├── composables/            # 打印相关组合式函数
│   │   │   ├── useInputValidation.js  # 输入验证
│   │   │   ├── usePrintConfig.js      # 打印配置管理
│   │   │   └── usePrintExecution.js   # 打印执行逻辑
│   │   ├── dialogs/                # 打印相关对话框
│   │   │   ├── PrintCopiesDialog.vue   # 打印份数对话框
│   │   │   ├── PrintPreviewDialog.vue  # 打印预览对话框
│   │   │   └── PrintProgressDialog.vue # 打印进度对话框
│   │   └── shared/                 # 打印共享资源
│   │       └── print-common.css    # 打印组件通用样式
│   └── settings/                   # 设置组件
│       ├── PrintSettings.vue       # 打印设置组件
│       └── print-setting.css       # 打印设置样式
├── src/                            # 源代码
│   ├── utils/
│   │   ├── print.js                # 打印机 API 封装
│   │   ├── LabelImageData.js       # 标签图片数据
│   │   └── ErrorHandler.js         # 统一错误处理器
│   ├── constants/
```

```
46  │   │   └── constants.js          # 常量定义（条码类型、错误码等）
47  │   ├── examples/
48  │   │   └── quickStart.js         # 快速开始示例
49  ├── uni_modules/                  # UniApp 插件
50  │   └── niimbot-jcsdk/            # 精臣打印机 SDK
51
```

## 打印机支持

| 支持打印机型号 | 插件Android侧支持通讯方式 | 插件IOS侧支持通讯方式 | SDK通讯协议类型 |
| --- | --- | --- | --- |
| H1 | 蓝牙（SPP） | 蓝牙（BLE） | 精臣私有协议 |
| H1S | 蓝牙（SPP） | 蓝牙（BLE） | |
| D11 | 蓝牙（SPP） | 蓝牙（BLE） | |
| D110 | 蓝牙（SPP） | 蓝牙（BLE） | |
| D101 | 蓝牙（SPP） | 蓝牙（BLE） | |
| B16 | 蓝牙（SPP） | 蓝牙（BLE） | |
| D110_M | 蓝牙（SPP） | 蓝牙（BLE） | |
| D11_H | 蓝牙（SPP） | 蓝牙（BLE） | |
| B21 | 蓝牙（SPP） | 蓝牙（BLE） | |
| B21S | 蓝牙（SPP） | 蓝牙（BLE） | |
| B21_Pro | 蓝牙（SPP） | 蓝牙（BLE） | |
| B11 | 蓝牙（SPP） | 蓝牙（BLE） | |
| B1 | 蓝牙（SPP） | 蓝牙（BLE） | |
| B203 | 蓝牙（SPP） | 蓝牙（BLE） | |
| B3S | 蓝牙（SPP） | 蓝牙（BLE） | |
| B3S_P | 蓝牙（SPP） | 蓝牙（BLE） | |
| B31 | 蓝牙（SPP） | 蓝牙（BLE） | |
| B4 | 蓝牙（SPP） | 蓝牙（BLE） | |
| | | | |

| | | |
|---|---|---|
| N1(B18) | 蓝牙（SPP） | 蓝牙（BLE） |
| B50&B50W | 蓝牙（SPP） | 蓝牙（BLE） |
| B32&B32R&Z401 | 蓝牙（SPP） | 蓝牙（BLE） |
| M2_H | 蓝牙（SPP） | 蓝牙（BLE） |
| M3 | 蓝牙（SPP） | 蓝牙（BLE） |
| K2 | 蓝牙（SPP） | 蓝牙（BLE） |
| K3 | 蓝牙（SPP） | 蓝牙（BLE） |
| K3_W | 蓝牙（SPP）、WIFI | 蓝牙（SPP）、WIFI |

# 一、使用说明

## 1.1 使用插件需要申请对应蓝牙权限

代码块

```
/** Android **/
<uses-permission android:name="android.permission.BLUETOOTH"/>
<uses-permission android:name="android.permission.BLUETOOTH_ADMIN"/>
<uses-permission
android:name="android.permission.ACCESS_FINE_LOCATION"/>

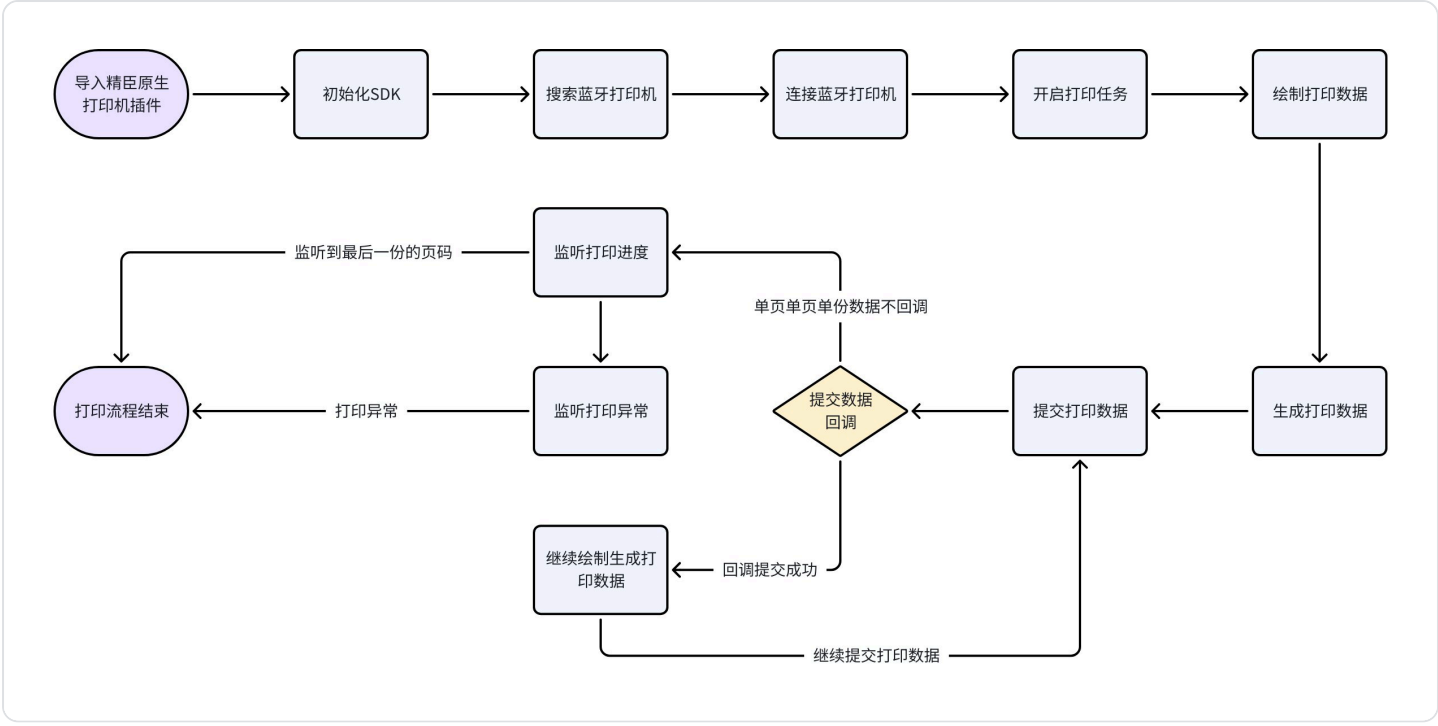
<!--WIFI相关-->
<uses-permission android:name="android.permission.ACCESS_FINE_LOCATION"/>
<uses-permission android:name="android.permission.INTERNET"/>

/** IOS **/
//IOS13 以下蓝牙打印机相关权限
NSBluetoothPeripheraUsageDescription

// IOS13 及更高版本蓝牙打印机相关权限
NSBluetoothAlwaysUsageDescription
// WIFI 打印机相关权限
NSLocalNetworkUsageDescription
```

## 1.2 接入示例：（详细使用方法可以参照插件示例项目）

### 1.2.1 蓝牙打印流程




**注意事项：**

1. 导入精臣原生打印机插件及初始化 SDK 仅需进行一次
2. 单页单份数据提交，数据提交接口不返回回调
3. 如打印页面与设备连接页面分开，需要对原生插件进行全局注册

### 1.2.2 蓝牙打印代码示例

代码块

```
// 引入niimbot-jcsdk中的JCPrinter类
import { JCPrinter } from '@/uni_modules/niimbot-jcsdk';

// 实例化JCPrinter
const jcapi = new JCPrinter();

// 初始化SDK
jcapi.initSDK()

// 搜索蓝牙设备
// 需要开发者先获取蓝牙和定位权限（部分Android版本需要定位权限才能使用蓝牙）
jcapi.getBluetoothDevices(function(r) {
    // 会搜索附近所有蓝牙设备，可以根据名称自行筛选过滤
    console.log("device:" + JSON.stringify(r))
})



// 蓝牙连接打印机
jcapi.openPrinterByDevice({
```

```
        address: item.address,
        name: item.name
    }, function(r) {
        if(r.code == 0) { // 连接成功 }
    })


    // 监听打印页码回调
    jcapi.didReadPrintCountInfo(function(r) { console.log(r) })

    // 监听打印错误回调
    jcapi.didReadPrintErrorInfo(function(r) { console.log(r) })



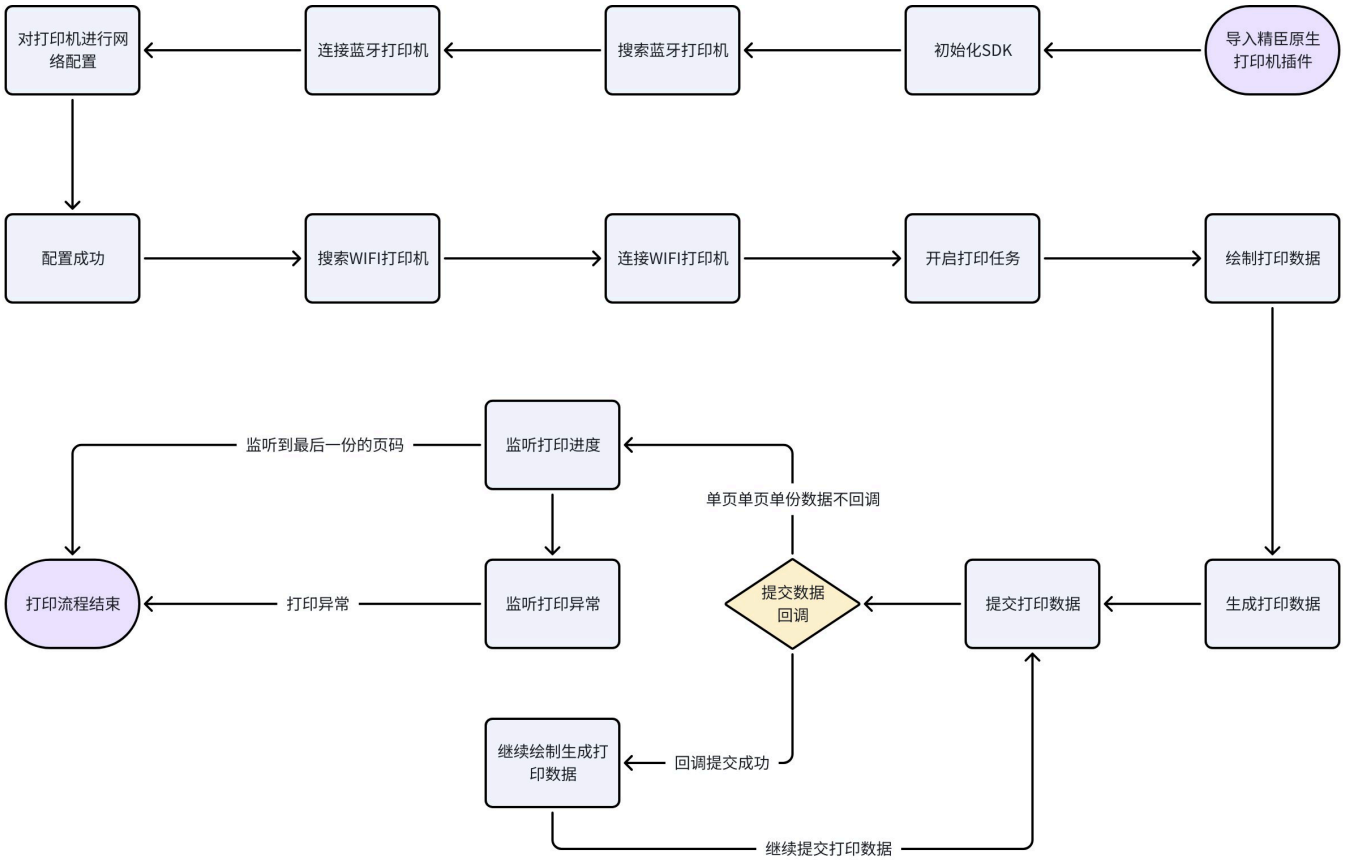

    // 设置打印任务
    jcapi.startJob({
        totalCount: 1,
        density: 5,
        labelType: 2,
        printMode: 1,
    },function(r){
        if(r.code == 0) {
          // 设置成功，开始绘制打印数据
          // 初始化画板
          jcapi.initDrawingBoard({ width: 40, height: 20, rotate: 0 });

          // 绘制文字
          jcapi.drawLabelText({
              x: 2,
              y: 2,
              width: 30,
              height: 20,
              value: "打印插件",
              fontSize: 4,
              lineMode: 1,
                          lineSpace: 0,
                    letterSpace: 0,
              roate: 0,
              textAlignHorizontal: 1,
              textAlignVertical: 1,
          })

          // 生成打印数据
                              let imageJsonObj = jcapi.generateLabelJson()
```

```
        // 发送打印数据
        jcapi.printData(imageJsonObj, {"printQuantity": 1,}, function(r) {
            if (r.code == 0) { console.log("打印成功") }
        })
    }
})
```

### 1.2.3 WIFI 打印流程




**注意事项：**

1. 导入精臣原生打印机插件及初始化 SDK 仅需进行一次
2. 打印机首次进行 WIFI 打印，需要先通过蓝牙进行连接配网，网络配置成功后，才可以进行网络打印机搜索，连接
    a. 目前仅支持 K3_W 机型
    b. 仅支持 2.4GWIFI 网络
    c. 手机（PDA）需要与打印机在同一网络
3. 单页单份数据提交，数据提交接口不返回回调
4. 如打印页面与设备连接页面分开，需要对原生插件进行全局注册

### 1.2.4 WIFI 打印代码示例（仅支持 K3_W）

代码块

```
// 导入原生插件js接口
const jcapi = uni.requireNativePlugin("JCSDK-JCApiModule")

// 初始化SDK
jcapi.initSDK()

// 搜索蓝牙设备
// 需要开发者先获取蓝牙和定位权限（部分Android版本需要定位权限才能使用蓝牙）
jcapi.getBluetoothDevices(function(r) {
    // 会搜索附近所有蓝牙设备，可以根据名称自行筛选过滤
    console.log("device:" + JSON.stringify(r))
})

// 蓝牙连接打印机
jcapi.openPrinterByDevice({
    address: item.address,
    name: item.name
}, function(r) {
    if(r.code == 0) { // 连接成功 }
})


// 蓝牙配网
jcapi.configWifi({
    wifiName:"wifi名称", //只支持2.4GHz
    wifiPwd:"wifi密码",
  }, function(r){
    if (r.code == 0) { //success}
  }
});

// 搜索局域网打印机
jcapi.getWifiDevices(function(r){
        console.log("device:" + JSON.stringify(r))
})

// 连接局域网打印机
jcapi.openPrinterByDevice({
    address: item.address,
    name: item.name
    deviceType: 1
}, function(r) {
    if(r.code == 0) { // 连接成功 }
```

```
})


// 监听打印页码回调
jcapi.didReadPrintCountInfo(function(r) { console.log(r) })

// 监听打印错误回调
jcapi.didReadPrintErrorInfo(function(r) { console.log(r) })

// 初始化画板
jcapi.initDrawingBoard({ width: 40, height: 20, rotate: 0 });

// 绘制文字
jcapi.drawLabelText({
    x: 2,
    y: 2,
    width: 30,
    height: 20,
    value: "打印插件",
    fontSize: 4,
    lineMode: 1,
                lineSpace: 0,
          letterSpace: 0,
    roate: 0,
    textAlignHorizontal: 1,
    textAlignVertical: 1,
})

// 生成打印数据
let imageJsonObj = jcapi.generateLabelJson()

// 设置打印任务
jcapi.startJob({
    totalCount: 1,
    density: 5,
    labelType: 2,
    printMode: 1,
},function(r){
    if(r.code == 0) { // 设置成功，开始发送打印数据
        // 发送打印数据
        jcapi.printData(imageJsonObj, {"printQuantity": 1,}, function(r) {
                  if (r.code == 0) { console.log("打印成功") }
        })
    }
})
```

# 二、接口说明

## 2.1 插件引入

代码块

```
// 引入插件
import {JCPrinter} from '@/uni_modules/niimbot-jcsdk';
// 实例化插件API
const jcapi = new JCPrinter();
```

## 2.2 基础功能

### 2.2.1 初始化 SDK

代码块

```
/**
 * 初始化SDK
 * @description 该方法用于初始化SDK，是在使用SDK之前必须执行的操作。
 */
function initSDK()
```

### 2.2.2 搜索打印机

#### 2.2.2.1 搜索所有蓝牙设备

代码块

```
/**
 * 搜索打印机（返回搜索到的所有蓝牙设备）
 * @description iOS只有name字段有效
 *
 * @param {Function} callback - 蓝牙回调，用于接收搜索结果
 * @param {Object[]} callbackResult - 搜索到的蓝牙设备信息数组
 * @param {string} callbackResult[].name - 蓝牙设备名称
 * @param {string} callbackResult[].address - 蓝牙设备MAC地址
 * @param {string} [callbackResult[].rssi] - 蓝牙设备的信号强度
 * @param {string} [callbackResult[].deviceType] - 蓝牙设备的信号强度
 * @returns {void}
 */
function getBluetoothDevices(callback)
```

#### 2.2.2.2 搜索指定类型蓝牙设备

代码块

```
/**
 * 搜索过滤指定打印机类型
 * @description iOS只有name字段有效
 *
 * @param {Object} data - 过滤参数对象
 * @param {string[]} data.name - 过滤名称，数组类型数据，可以设置多个过滤。通过蓝牙名称前缀过滤，不区分大小写。
 * @param {Function} callback - 蓝牙回调，用于接收过滤后的搜索结果
 * @param {Object[]} callbackResult - 过滤后的蓝牙设备信息数组
 * @param {string} callbackResult[].name - 蓝牙设备名称
 * @param {string} callbackResult[].address - 蓝牙设备MAC地址
 * @param {string} [callbackResult[].rssi] - 蓝牙设备的信号强度
 * @returns {void}
 */
function getBluetoothDevicesWithName(data, callback)
```

#### 2.2.2.3 搜索局域网打印机

代码块

```
/**
 * 搜索局域网打印机
 * @description 局域网打印机没有rssi数值
 *
 * @param {Function} callback - 搜索结果回调，用于接收局域网打印机的信息
 * @param {Object[]} callbackResult - 局域网打印机的搜索结果数组
 * @param {string} callbackResult[].name - 打印机名称
 * @param {string} callbackResult[].address - 打印机网络地址
 * @param {string} [callbackResult[].rssi] - 打印机的信号强度（可选，
 * @returns {void}
 */
function getWifiDevices(callback)
```

### 2.2.3 连接打印机

代码块

```
/**
 * 连接打印机
 *
 * @param {Object} data - 连接参数对象
 * @param {string} data.address - 打印机的蓝牙MAC地址或局域网打印机的IP地址
```

```
 * @param {string} data.name - 打印机的蓝牙名称（局域网打印机可为空）
 * @param {number} data.deviceType - 打印机类型
 *                                   0: 蓝牙打印机
 *                                   1: 局域网打印机
 * @param {Function} callback - 连接回调函数
 * @param {Object} callbackResult - 回调结果对象
 * @param {number} callbackResult.code - 连接结果代码
 *                                     0: 连接成功
 *                                    -1: 连接失败
 * @returns {void}
 */
function openPrinterByDevice(data, callback)
```

### 2.2.4 断开连接

代码块

```
/**
 * 断开打印机连接
 */
function close()
```

### 2.1.5 打印异常回调

代码块

```
/**
 * 监听当前任务打印异常回调
 * @callback callback - 异常回调函数
 * @param {Object} callbackResult - 异常信息对象
 * @param {number} callbackResult.code - 错误类型代码，表示打印异常的类型
 * @returns {void}
 *
 * 异常类型:
 * 1-盒盖打开
 * 2-缺纸
 * 3-电量不足
 * 4-电池异常
 * 5-手动停止（按键）
 * 6-数据错误
 * 7-温度过高
 * 8-出纸异常
 * 9-正在打印
 * 10-没有检测到打印头
 * 11-环境温度过低
 * 12-打印头未锁紧
```

```
 * 13-未检测到碳带
 * 14-不匹配的碳带
 * 15-用完的碳带
 * 16-不支持的纸张类型
 * 17-纸张类型设置失败
 * 18-打印模式设置失败
 * 19-设置浓度失败
 * 20-写入rfid失败
 * 21-边距设置失败
 * 22-通讯异常
 * 23-打印机连接断开
 * 24-画板参数错误
 * 25-旋转角度错误
 * 26-json参数错误
 * 27-出纸异常（B3S）
 * 28-检查纸张类型
 * 29-RFID标签未进行写入操作
 * 30-不支持浓度设置
 * 31-不支持的打印模式
 * 34-不支持RFID写入
 * 50-非法标签
 * 51-非法碳带和标签
 * 52-固件接收数据超时
 * 53-非专用色带（当前机型使用了型号不匹配的色带）
 * 58-非正品耗材（标签与色带均未识别）（适用于部分新机型）
 * 59-非正品色带（色带均未识别）（适用于部分新机型）
 * 60-耗材超限（标签与色带均超过使用限制）（适用于部分新机型）
 * 61-色带超限（色带超过使用限制）（适用于部分新机型）
 * 62-非正品标签（标签未识别）（适用于部分新机型）
 * 63-标签超限（标签超过使用限制）（适用于部分新机型）
 */
function didReadPrintErrorInfo(callback)
```

### 2.2.6 打印进度回调

代码块

```
/**
 * 当前任务打印进度回调
 *
 * @callback callback - 打印进度回调函数
 * @param {Object} callbackResult - 打印进度信息对象
 * @param {number} callbackResult.count - 已打印标签数
 * @returns {void}
 */
```

```
function didReadPrintCountInfo(callback)
```

### 2.2.7 蓝牙配网

代码块

```
/**
 * 打印机配网接口
 *
 * @description 该方法用于在蓝牙连接下给打印机进行配网操作，以连接到指定的Wi-Fi网络。
 *
 * @param {Object} data - 配网参数的对象。
 * @param {string} data.wifiName - Wi-Fi网络的名称（只支持2.4GHz网络）。
 * @param {string} data.wifiPwd - Wi-Fi网络的密码。
 *
 * @param {function} callback - 一个回调函数，用于接收配网操作的结果。
 * @param {Object} callback.result - 一个包含配网结果信息的对象。
 * @param {number} callback.result.code - 配网操作的结果码：
 *   - 0: 配网成功
 *   - -1: 配网失败
 *   - -3: 不支持配网
 *
 * 注意：只能在蓝牙连接下给打印机进行配网操作，无法在Wi-Fi连接下进行。
 * 另外，只有部分机型支持局域网打印，且只支持2.4GHz的网络。
 *
 * @example
 * configWifi({ wifiName: 'MyWiFiNetwork', wifiPwd: 'MyWiFiPassword' }, function(result) {
 *   if (result.code === 0) {
 *     console.log('配网成功');
 *   } else if (result.code === -1) {
 *     console.error('配网失败');
 *   } else if (result.code === -3) {
 *     console.warn('不支持配网');
 *   }
 * });
 *
 * @returns {void} - 该方法通常没有返回值，配网结果会通过回调函数传递。
 */
function configWifi(data, callback)
```

### 2.2.8 连接状态判断

代码块

```
/**
```

```
 * 检测设备连接
 * @return {JSONObject}
 *  {'value': 0}  // 0: 没有连接设备，  1: 有连接设备
 */
function isConnected()
```

## 2.3 绘制

### 2.3.1 初始化绘制

代码块

```
/**
 * 初始化绘制
 *
 * @param {Object} data - 绘制参数对象
 * @param {number} data.width - 画板宽度，单位mm
 * @param {number} data.height - 画板高度，单位mm
 * @param {number} [data.rotate=0] - 画板旋转角度（可选），默认为0，不旋转。支持的旋转角度：0、90、180、270。
 * @returns {void}
 */
function initDrawingBoard(data)
```

### 2.3.2 绘制文本

代码块

```
/**
 * 绘制文本
 * @param {{}} data 绘制参数
 * @param {number} data.x 元素起始x坐标
 * @param {number} data.y 元素起始y坐标
 * @param {number} data.width 文本框宽度，单位mm
 * @param {number} data.height 文本框高度，单位mm
 * @param {string} data.value 绘制文字内容
 * @param {number} data.fontSize 字体大小，单位mm。默认高度4mm。
 * @param {number} data.rotate（可选）画板旋转角度，默认为0，不旋转。支持的旋转角度0、90、180、270。
 * @param {number} data.lineMode 换行方式，默认为2。
 *                                  1:宽高固定，内容大小自适应（字号/字符间距/行间距，按比例缩放）
 *                                  2:宽度固定，高度自适应
 *                                  6.宽高固定，内容超过预设的宽高时自动缩小
 * @param {number} data.letterSpace 字间隔。默认为0。
 * @param {number} data.lineSpace 行间隔。默认为0。
```

```
 * @param {number} data.textAlignHorizontal 文字水平布局，默认为1。
 *                                          0：左对齐
 *                                          1：水平居中
 *                                          2：右边对齐
 * @param {number} data.textAlignVertical 文字垂直布局，默认为1。（备注：换行模式为2时无效）
 *                                          0：顶部对齐
 *                                          1：垂直居中
 *                                          2：底部对齐
 * @param {number} data.bold 文字加粗样式设置，默认为0。
 *                                          0：正常
 *                                          1：加粗
 * @param {number} data.italics 文字斜体样式设置，默认为0。
 *                                          0：正常
 *                                          1：斜体
 * @param {number} data.underline 文字下划线样式设置，默认为0。
 *                                          0：正常
 *                                          1：下划线
 *
 */
function drawLabelText(data)
```

### 2.3.3 绘制条形码

代码块

```
/**
 * 绘制条形码
 * @param {{}} data 绘制参数
 * @param {number} data.x 元素起始x坐标
 * @param {number} data.y 元素起始y坐标
 * @param {number} data.width 条码宽度，单位mm
 * @param {number} data.height 条码高度，单位mm
 * @param {string} data.value 条码内容
 * @param {number} data.fontSize 字体大小，单位mm。默认4mm。
 * @param {number} data.roate（可选）画板旋转角度，默认为0，不旋转。支持的旋转角度 0、90、180、270。
 * @param {number} data.codeType 条码类型。默认类型20。
 *                              20:CODE128
 *                              21:UPC-A
 *                              22:UPC-E
 *                              23:EAN8
 *                              24:EAN13
 *                              25:CODE93
 *                              26:CODE39
 *                              27:CODEBAR
 *                              28:ITF25
```

```
21    * @param {number} data.textHeight 文本高度。默认高度4mm。
22    * @param {number} data.textPosition 条码文本位置。默认为0。
23    *                         0: 下方显示
24    *                         1: 上方显示
25    *                         2: 不显示
26    */
27   function drawLabelBarCode(data)
```

### 2.3.4 绘制二维码

代码块

```
1   /**
2    * 绘制二维码
3    * @param {{}} data 绘制参数
4    * @param {number} data.x 元素起始x坐标
5    * @param {number} data.y 元素起始y坐标
6    * @param {number} data.width 二维码宽度，单位mm
7    * @param {number} data.height 二维码高度，单位mm
8    * @param {string} data.value 二维码内容
9    * @param {number} data.rotate （可选）画板旋转角度，默认为0，不旋转。支持的旋转角度
     0、90、180、270。
10   * @param {number} data.codeType 二维码类型，默认类型31。
11   *
                                        31:QR_CODE
12   *
                                        32:PDF417
13   *
                                        33:DATA_MATRIX
14   *
                                        34:AZTEC
15   */
16  function drawLabelQrCode(data)
```

### 2.3.5 绘制线条

代码块

```
1   /**
2    * 绘制线条
3    * @param {{}} data 绘制参数
4    * @param {number} data.x 元素起始x坐标
5    * @param {number} data.y 元素起始y坐标
6    * @param {number} data.width 线条宽度，单位mm
7    * @param {number} data.height 线条高度，单位mm
```

```
 * @param {number} data.rotate（可选）画板旋转角度，默认为0，不旋转。支持的旋转角度0、90、180、270。
 * @param {number} data.lineType 线条类型，默认类型为1。
 *                                                          1: 实线 , 2:虚线
 */
function drawLabelLine(data)
```

### 2.3.6 绘制形状

代码块

```
/**
 * 绘制形状
 * @param {{}} data 绘制参数
 * @param {number} data.x 元素起始x坐标
 * @param {number} data.y 元素起始y坐标
 * @param {number} data.width 形状宽度，单位mm
 * @param {number} data.height 形状高度，单位mm
 * @param {number} data.graphType 图像类型，默认类型为1。1: 圆形，2: 椭圆， 3:矩形， 4: 圆角矩形
 * @param {number} data.rotate（可选）画板旋转角度，默认为0，不旋转。支持的旋转角度0、90、180、270。
 * @param {number} data.cornerRadius 圆角半径，单位mm
 * @param {number} data.lineWidth 线条宽，单位mm。默认线宽为0.2mm。
 * @param {number} data.lineType 线条类型，默认类型为1。1: 实线，2: 虚线
 */
drawLabelGraph(data)
```

### 2.3.7 绘制图片

代码块

```
/**
 * 绘制图片
 * @param {{}} data 绘制参数 需要去除base64数据头
 * @param {string} data.imageData 图片base64数据，需要去除base64数据头的前缀
 * @param {number} data.x 元素起始x坐标
 * @param {number} data.y 元素起始y坐标
 * @param {number} data.width 图片宽度，单位mm
 * @param {number} data.height 图片高度，单位mm
 * @param {number} data.rotate（可选）画板旋转角度，默认为0，不旋转。支持的旋转角度0、90、180、270。
 * @param {number} data.imageProcessingType （可选）处理算法，默认为1
 * @param {number} data.imageProcessingValue （可选）阈值，默认127
 */
```

```
13  function drawLabelImage(data)
```

### 2.3.8 生成绘制数据

此接口生成的数据为 `printData` 接口提交的打印数据。

代码块

```
1   /**
2    * 生成绘制内容
3    * @return {{}} 绘制内容json对象
4    */
5   function generateLabelJson()
```

### 2.3.9 预览数据

代码块

```
1   /**
2    * 预览数据
3    *
4    * @description 该方法用于生成预览图片，根据接口“generateLabelJson()”返回的数据。
5    *
6    * @param {Object} data - 包含接口“generateLabelJson()”返回的数据的对象。
7    *
8    * @param {Object} returnObject - 返回一个包含预览图片的对象。
9    * @param {string} returnObject.base64 - 预览图片的Base64编码。
10   *    - 成功：包含预览图片的Base64编码。（不包含Base64数据头）
11   *    - 失败：一个空字符串。
12   *
13   * @example
14   * const labelData = generateLabelJson();
15   * const imageData =
     ""+generatePrintPreviewImage(labelData);
16   *
17   * @returns {void} - 该方法通常没有返回值，预览图片信息会通过回调函数传递。
18   */
19  function generatePrintPreviewImage(data)
```

## 2.4 打印

### 2.4.1 设置打印任务

代码块

```
1   /**
```

```
 * 设置打印任务
 * @param {Object} config - 打印任务参数对象
 * @param {number} config.totalCount - 总打印份数，表示所有页面的打印份数之和。例如，如果你有3页需要打印，第一页打印3份，第二页打印2份，第三页打印5份，那么totalCount的值应为10（3+2+5）。
 * @param {number} config.density -  D11、D101、D110、H10、B16、B18、N1 1~3 默认2
 *                       B3S、B203、B1、B203、B31、B4、K3、K3W、K2、M2、M3 1~5 默认3
 *                       B50、B11、B50W、B32、Z401 1~15 默认8
 * @param {number} config.labelType - 标签类型
 *   1: 间隙纸
 *   2: 黑标纸
 *   3: 连续纸
 *   4: 定孔纸
 *   5: 透明纸
 *   6: 标牌
 *   10: 黑标间隙纸
 * @param {number} config.printMode - 打印模式
 *   1: 热敏  D11、D101、D110、H10、B16、B3S、B203、B1、B203、B31、B4、K3、K3W、B11仅支持热敏
 *   2: 热转印 B50、 B50W、B32、Z401、M2、M3、N1、B18仅支持热转印
 * @param {Function} callback - 设置打印任务回调
 * @param {Object} callbackResult - 回调结果对象
 * @param {number} callbackResult.code - 设置打印任务的结果代码
 *   - 0: 成功
 *   - -1: 失败
 * @returns {void}
 */
function startJob(config, callback)
```

### 2.4.2 提交打印数据

代码块

```
/**
 * 提交打印数据
 * @param {Object} imageJson - 打印数据，使用函数 generateLabelJson() 返回的数据
 * @param {Object} printQuantity - 用于指定当前页的打印份数，需传入包含 printQuantity 属性的对象
 *   格式示例：{ printQuantity: 1 }
 *   例如：如果你需要打印3页，第一页打印3份，第二页打印2份，第三页打印5份，
 *   那么在3次提交数据时，printQuantity参数值分别应为 { printQuantity: 3 }、{ printQuantity: 2 }、{ printQuantity: 5 }
 * @param {Function} callback - 数据提交回调
 * @param {Object} callbackResult - 回调结果对象
 * @param {number} callbackResult.code - 提交数据的结果代码
 *   - 0: 成功（仅代码数据提交成功，非打印成功）
```

```
 *   - -1: 失败
 * @returns {void}
 */
function printData(imageJson, printQuantity, callback)
```

同一个打印任务中需要打印多页数据时，需要等待上一个打印数据的成功回调再继续发送。

### 2.4.3 取消打印

用于打印过程中取消打印

代码块

```
/**
 * 取消打印任务
 * @param {Function} callback - 数据提交回调
 * @param {Object} callbackResult - 回调结果对象
 * @param {number} callbackResult.code - 提交数据的结果代码
 *   - 0: 成功（仅代码数据提交成功，非打印成功）
 *   - -1: 失败
 * @returns {void}
 */
function cancelJob(callback)
```